

KEIN SQL, KEIN ORM, KEIN SCHLAF – EIN KEY- VALUE-STORE IN C++

Ein selbstbau-Abenteuer zwischen Sockets,
Logs und nächtlichen Debug-Sessions.

Wien @ Cloudflight, 23.06.2025

Stefan Mikic

AGENDA

- 01** Warum das ganze? 🤔
- 02** Wer bin ich? 🐱
- 03** Cloudflight ☁️
- 04** Die Idee und der erste Entwurf 💡
- 05** Logs, weil RAM vergisst 📄
- 06** Networking, aber bitte mit Sockets 🔌
- 07** Refactoring und Architekturentscheidungen 🏗️
- 08** Replikation, einfach aber fragil 📦
- 09** Leader-Election, wer ist der Boss? 👑
- 10** Learning Lessons 🤔
- 11** “Roadmap” 🗺️

01

WARUM DAS GANZE? 🤔

01 Warum das ganze? 🤔

***Wie schwer kann das
eigentlich sein?***

01 Warum das ganze? 🤔

***Es war schwerer als
gedacht, aber ich
würde es wieder tun***

02

WER BIN ICH?



02 Wer bin Ich?

*Daten sind mein Gemüse    – gesund, vielseitig
und manchmal schwer verdaulich, aber am Ende bringen sie immer Mehrwert!*

 <https://smikic.com>

 <https://github.com/stejul>

 **Heute**
TCP
Race Conditions
verlorene Logs



STEFAN MIKIC

 Data Engineer @ Cloudflight

 **Normalerweise**
Datawarehouses
Spark
Datenmodellierung

02 Wer bin Ich?



03

CLOUDFLIGHT 

What is this “Cloudflight”?
And what do we do?

Cloudflight –
a European
digital engineering
company focused
on co-creating
mission-critical
solutions.



1,000
Customers



600
Engineers



2,000
Projects

Trusted by our clients

Healthcare

Mobility

Manufacturing

Logistics

Retail

Public Sector

Aerospace & Defense

Always close to our colleagues

Cluj

Frankfurt

Graz

Hamburg

Innsbruck

Karlsruhe

Kiel

Linz

Munich

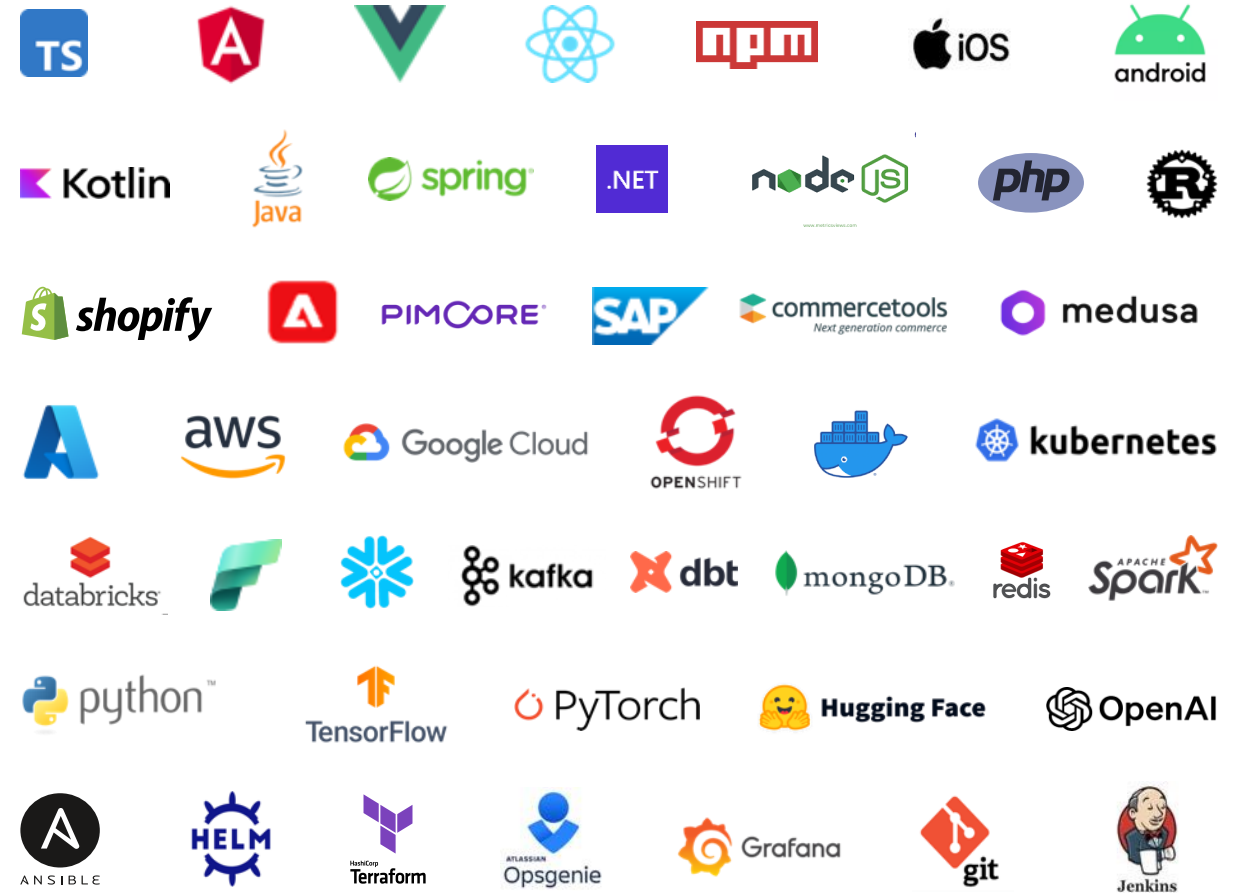
Wroclaw

Vienna

So what does Cloudflight actually do?



We use a wide range
of technologies



We provide services in four areas



Software Engineering



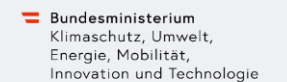
Digital Commerce



Cloud Solutions



Data & AI



04

DIE IDEE UND DER ERSTE ENTWURF



04 Die Idee und der erste Entwurf💡

***Ich wollte eigentlich
nur verstehen, wie
Datenbanken intern
funktionieren...***

04 Die Idee und der erste Entwurf💡

Ziel

Ein einfacher **PUT** / **GET** / **DEL** Speicher, alles im Ram und nichts mit Netzwerk oder Persistenz

04 Die Idee und der erste Entwurf💡

Umsetzung

Die In-Memory Datenbank wird eine vorhandene Hash-Map implementieren

```
kvstore.hpp
1  #pragma once
2  #include <unordered_map>
3  #include <string>
4  #include <optional>
5
6  class KVStore {
7  public:
8      void put(const std::string& key, const std::string& value);
9      std::optional<std::string> get(const std::string& key) const;
10     bool del(const std::string& key);
11 private:
12     std::unordered_map<std::string, std::string> store;
13 };
```

04 Die Idee und der erste Entwurf💡

Umsetzung

```
▲ Projects/distribkvstore/build ./kvstore  
GET hello: world  
deleted hello  
hello not found after deletion
```

04 Die Idee und der erste Entwurf💡

Herausforderung

*Was passiert bei
einem Neustart? 🤔*

04 Die Idee und der erste Entwurf💡

Herausforderung

Die Sprache generell...

Die Daten/Operationen werden nicht geloggt...

05

LOGS, WEIL RAM VERGISST 

05 Logs, weil RAM vergisst

Ziel

Unser KV-Store wird
mit einem **FileAppendLog** erweitert

Logger speichert unsere **PUT** / **DEL**
Operationen, danach in die Datenbank

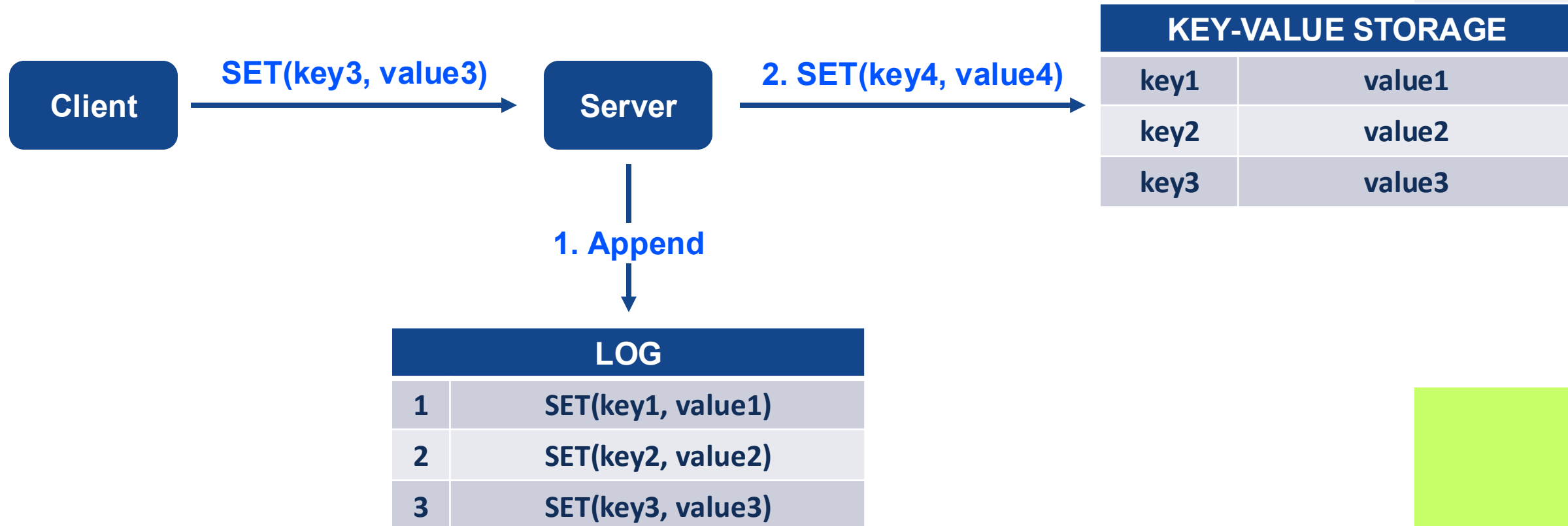
05 Logs, weil RAM vergisst

Warum Write-Ahead-Logging?

- **Verlieren unsere Änderungen nicht**
- **Möglichkeit nach Fehlern selbst zu heilen**
- **Replikation ermöglichen**

05 Logs, weil RAM vergisst

Warum Write-Ahead-Logging?



05 Logs, weil RAM vergisst 📁

Herausforderungen

flush() nicht vergessen 😡



06

NETWORKING, ABER BITTE MIT SOCKETS 

06 Networking, aber bitte mit Sockets

Ziel

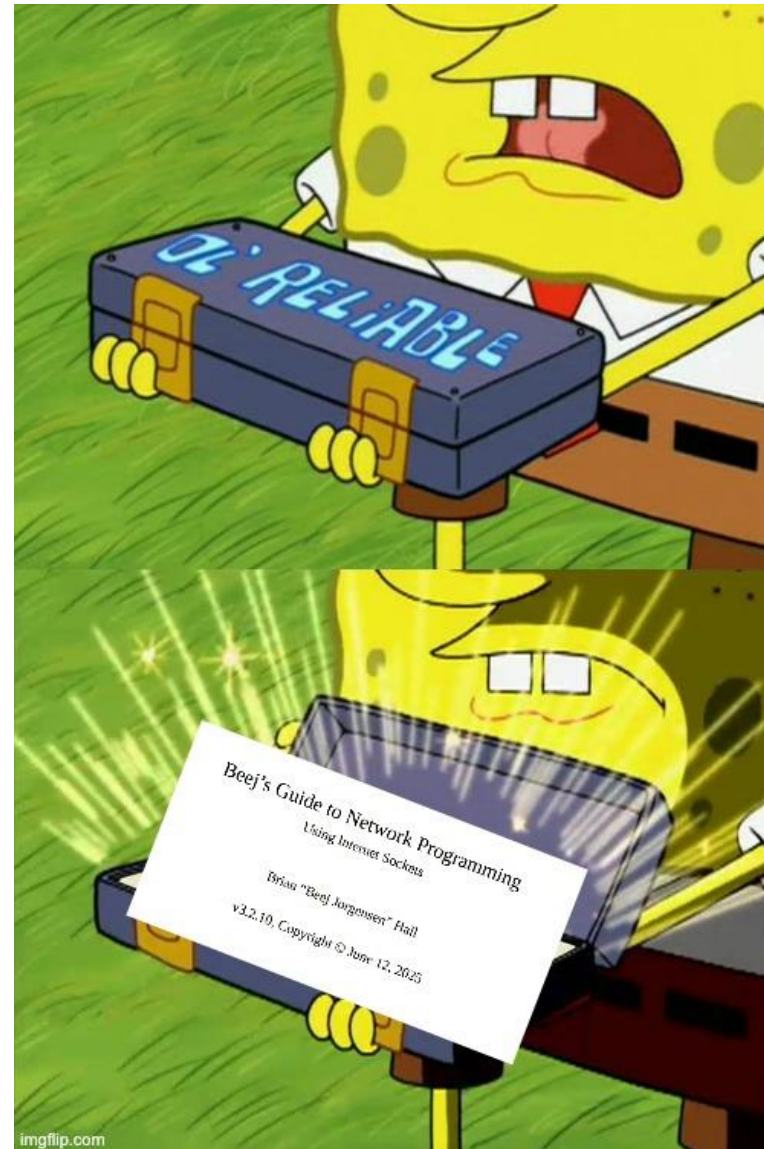
Was ist ein verteiltes System ohne einer Client-Server Architektur?

Jetzt wird es richtig ernst...

06 Networking, aber bitte mit Sockets 🍝

Ziel

**Zur meiner Rettung:
*Beej's Guide to
Network Programming***



06 Networking, aber bitte mit Sockets

Ziel

Nach etlichen Versuchen, *et voilà* 

```
▲ Projects/distribkvstore/build ./kvstore_server  
KVServer listening port 12345...
```

```
▲ Projects/distribkvstore/build ./kvstore_client 127.0.0.1 12345  
> PUT foo bar  
OK  
> GET foo bar  
bar  
> █
```

06 Networking, aber bitte mit Sockets

Herausforderung

**Nebenläufigkeit und Netzwerkprogrammierung
sind kein gutes Paar**

Protokolldefinition in der Netzworkkommunikation

07

REFACTORING UND ARCHITEKTURENTSCHEIDUNGEN



07 REFACTORING UND ARCHITEKTURENTSCHEIDUNGEN



Ziel

***Wenn der Code
schreit, bau ihm ein
Zuhause***

- Der Architekt, der zu spät kam

07 Refactoring und Architekturentscheidungen



Refactoring TL;DR

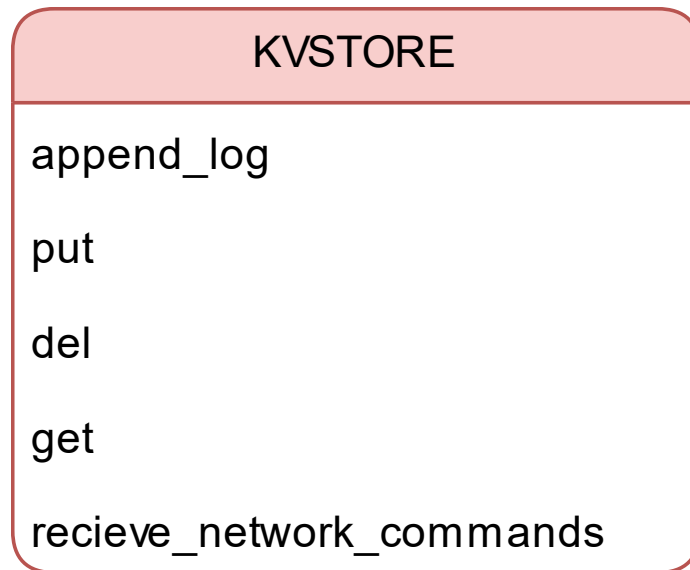
- **Die Kernlogik modularisieren**
- **Netzwerk-Kommunikation abstrahieren**
- **Cluster-Management als eigene Schicht**
- **Persistenz entkoppeln**

07 Refactoring und Architekturentscheidungen

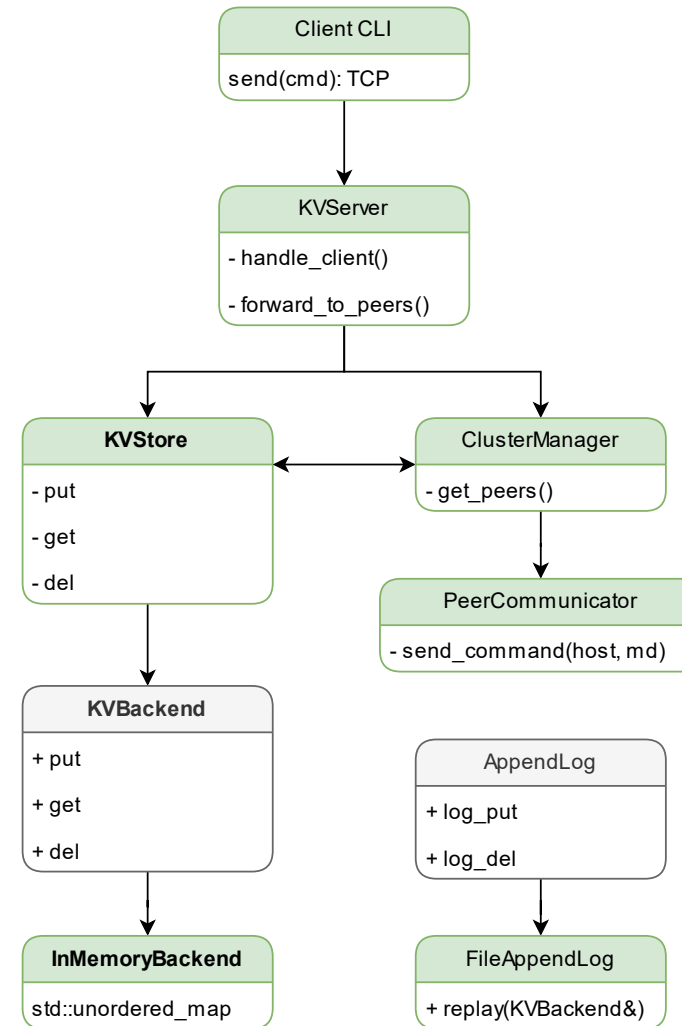


Refactoring TL;DR

Vorher



Nachher



Structure...

08

REPLIKATION, EINFACH ABER FRAGIL



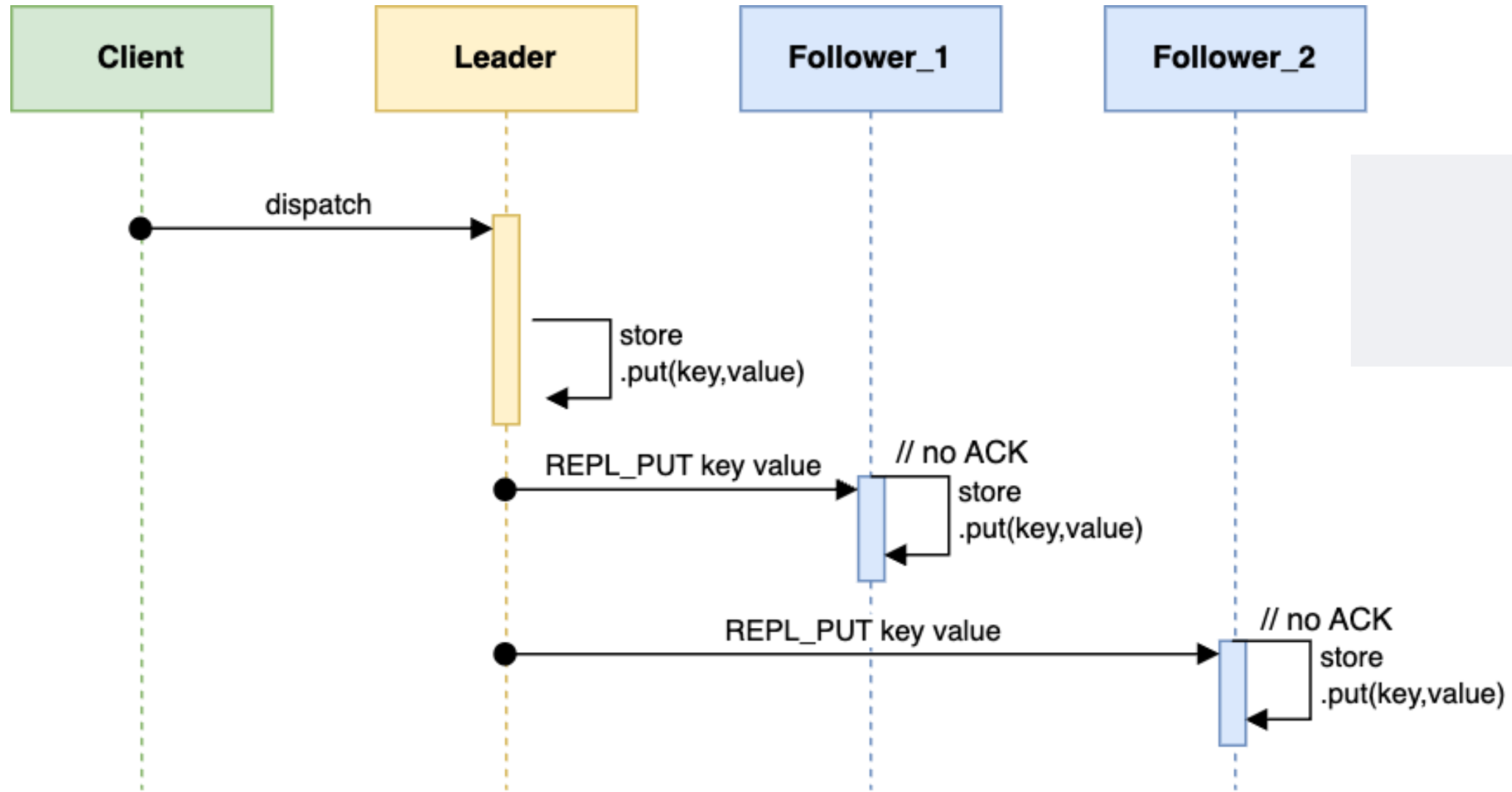
08 Replikation, einfach aber fragil

Ziel

Wenn ein Leader Daten empfängt, sollen alle Follower synchron bleiben.

08 Replikation, einfach aber fragil

Lösung



08 Replikation, einfach aber fragil

Implementierung

Das ganze implementieren wir mit
PeerCommunicator::send_command

```
peer_communicator.cpp
1 #include "peer_communicator.hpp"
2 #include <sys/socket.h>
3 #include <arpa/inet.h>
4 #include <unistd.h>
5 #include <iostream>
6 void PeerCommunicator::send_command(const std::string& host, int port, const std::string& cmd) {
7     int sock = socket(AF_INET, SOCK_STREAM, 0);
8     if (sock < 0) return;
9     sockaddr_in addr{};
10    addr.sin_family = AF_INET;
11    addr.sin_port = htons(port);
12    inet_pton(AF_INET, host.c_str(), &addr.sin_addr);
13    if (connect(sock, (struct sockaddr*)&addr, sizeof(addr)) < 0) {
14        close(sock);
15        return;
16    }
17    send(sock, cmd.c_str(), cmd.length(), 0);
18    close(sock);
19 }
```

08 Replikation, einfach aber fragil

Probleme

- **Keine Wiederholungsversuche bei Netzwerkfehlern**
- **Keine Checks, ob Follower noch leben**

09

LEADER-ELECTION, WER IST DER BOSS? 

09 LEADER-ELECTION, WER IST DER BOSS? 👑

Ziel

***Und plötzlich fragt
sich jeder: „Wer ist
hier eigentlich der
Chef?“***

09 LEADER-ELECTION, WER IST DER BOSS? 👑

Konsens-Algorithmus

Was ist Leader Election?

> *Ein Prozess, um die Zusammenarbeit mit verschiedenen Nodes zu vereinfachen.*

09 LEADER-ELECTION, WER IST DER BOSS? 👑

Konsens-Algorithmus

Warum ist das wichtig?

- **Fehlertoleranz verringern**
- **Konsistenz erhöhen**
- **Skalierbarkeit erhöhen**

09 LEADER-ELECTION, WER IST DER BOSS? 🏰

Konsens-Algorithmus - Einfache

Welche Algorithmen gibt es?

- ***Bully Algorithmus***
- ***Randomized (Lottery) Election***
- ***Ring Algorithm***

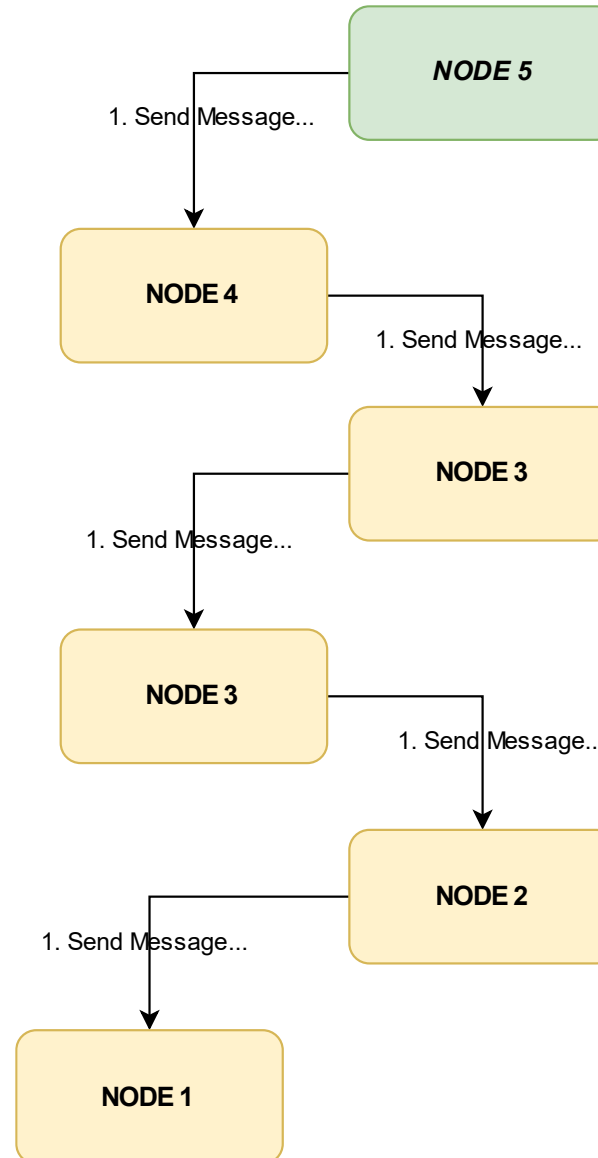
09 LEADER-ELECTION, WER IST DER BOSS? 🏰

Konsens-Algorithmus - Bully

- Jeder Node hat eine *eindeutige* ID (IP-Adresse, Node ID)
- Eine Rangordnung existiert: höhere ID = höherer Rang
- Node mit der höchsten ID = Leader

09 LEADER-ELECTION, WER IST DER BOSS? 🏰

Konsens-Algorithmus - Bully



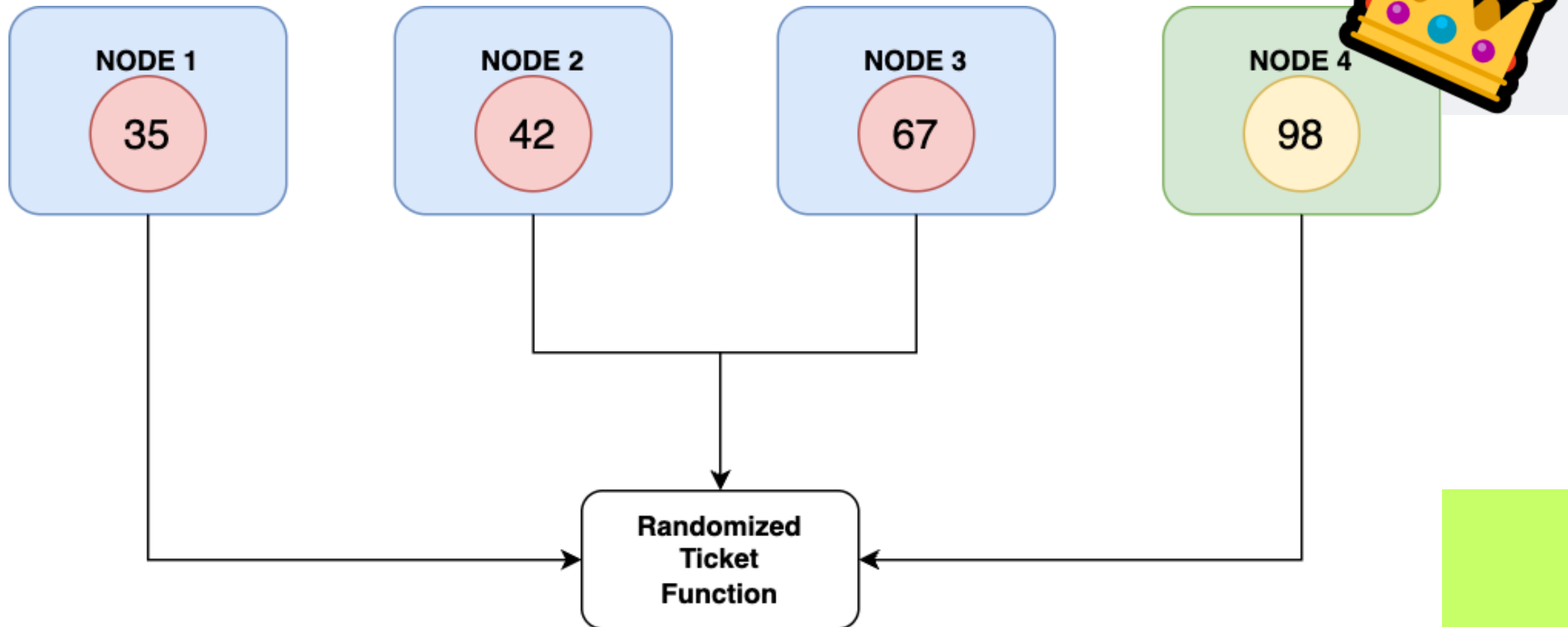
09 LEADER-ELECTION, WER IST DER BOSS? 🏰

Konsens-Algorithmus – Randomized (Lottery) Election

- Jeder Node *zieht ein zufälliges Los*
- Alle Lose werden miteinander verglichen
- Los mit der höchsten Zahl = Leader

09 LEADER-ELECTION, WER IST DER BOSS? 👑

Konsens-Algorithmus – Randomized (Lottery) Election



09 LEADER-ELECTION, WER IST DER BOSS? 👑

Konsens-Algorithmus - Schwere

Welche Algorithmen gibt es?

- *Paxos*
- *Raft*

09 LEADER-ELECTION, WER IST DER BOSS? 👑

Konsens-Algorithmus - Paxos

- **Konsensprotokoll aus mehreren Phasen**
 - 📧 **Proposal: *Node schlägt sich selbst als Leader vor***
 - ✅ **Acceptance: *Andere Nodes akzeptieren dies, wenn kein besserer Vorschlag vorhanden ist***

Jeder spricht darüber, kaum jemand versteht es

09 LEADER-ELECTION, WER IST DER BOSS?

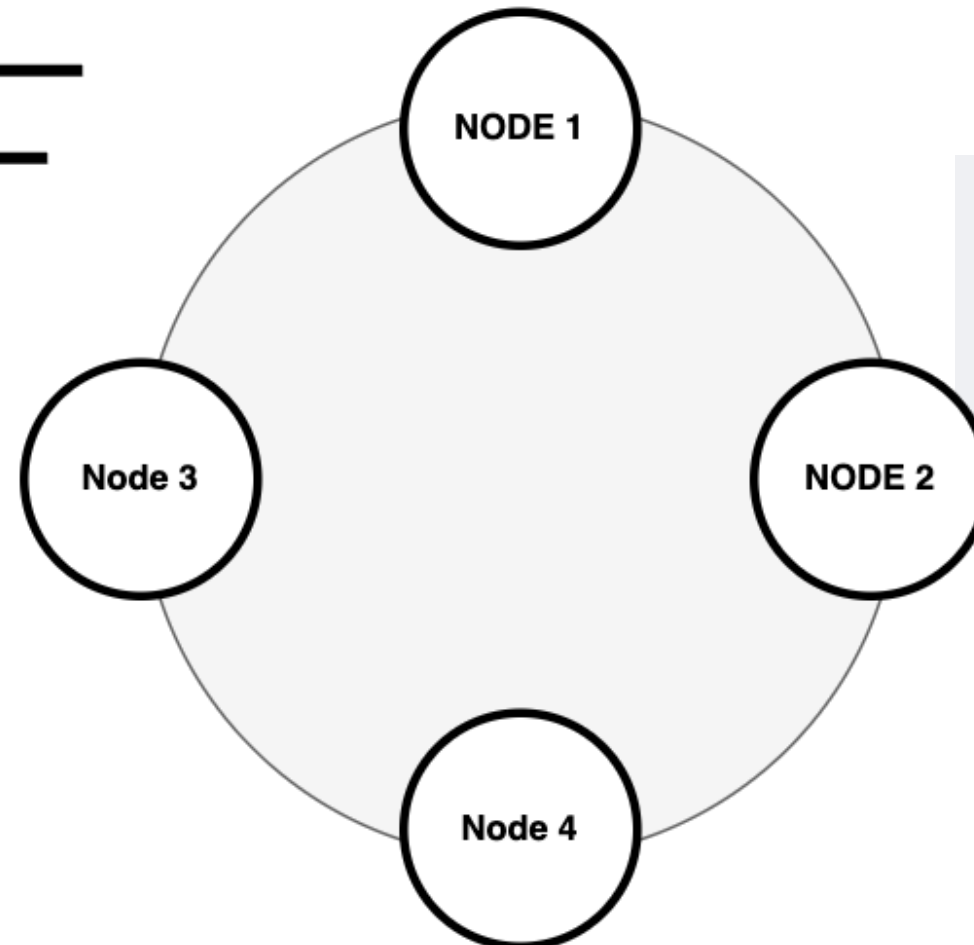
Konsens-Algorithmus – Raft

- Implementiert Leader-Election mit Log-Replikation 
- *Zeit wird in sog. Terms unterteilt* 
- Jeder *Term* beginnt mit einem *Leader-Wahl* 
- Gefolgt durch *Log-Replikation vom Leader an Follower*
- Bei Probleme wird *automatisch eine Neuwahl stattfinden*

09 LEADER-ELECTION, WER IST DER BOSS? 🏰

Konsens-Algorithmus – Raft

Follower 
Leader 
Candidate 



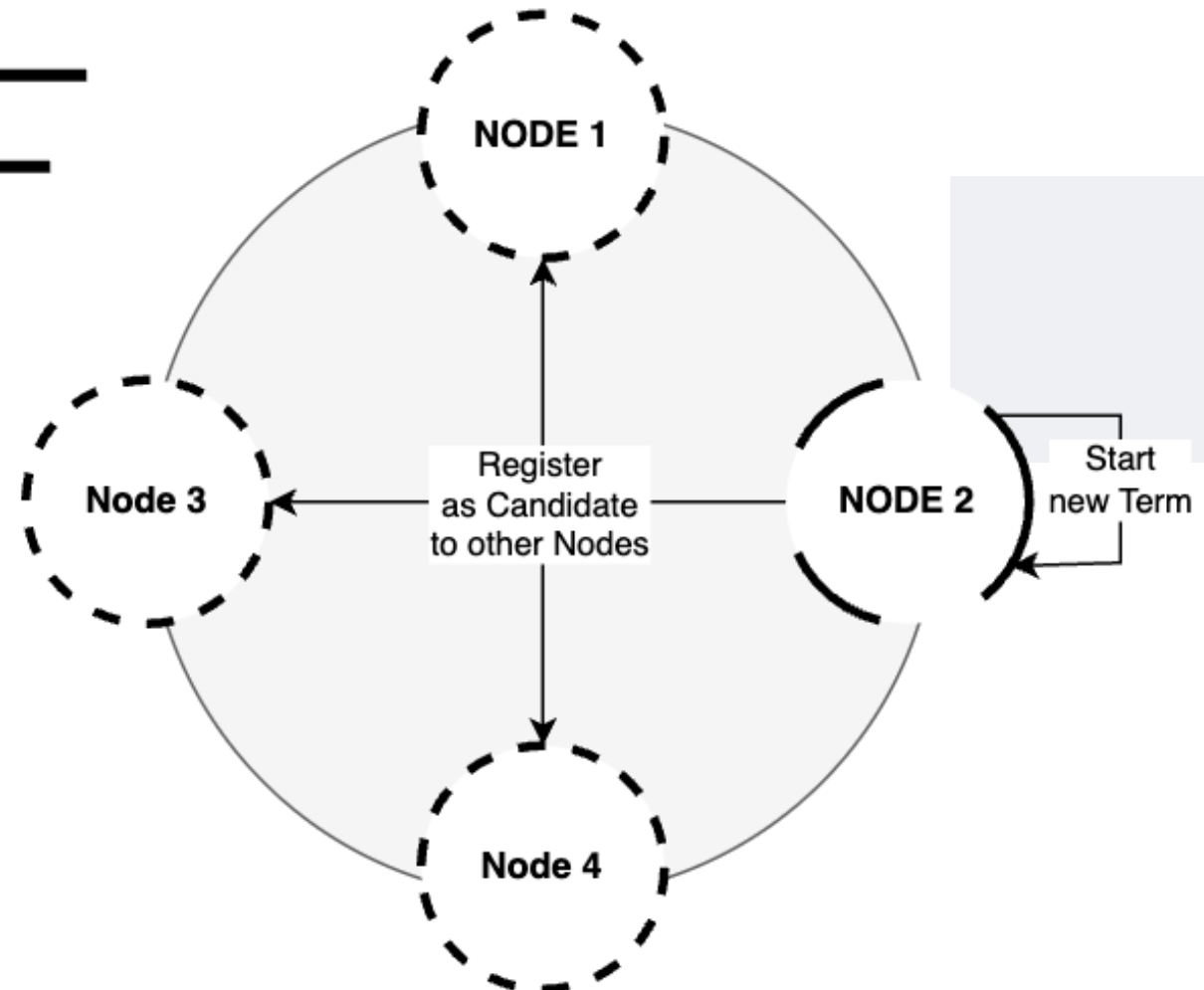
09 LEADER-ELECTION, WER IST DER BOSS? 👑

Konsens-Algorithmus – Raft

Follower 

Leader 

Candidate 



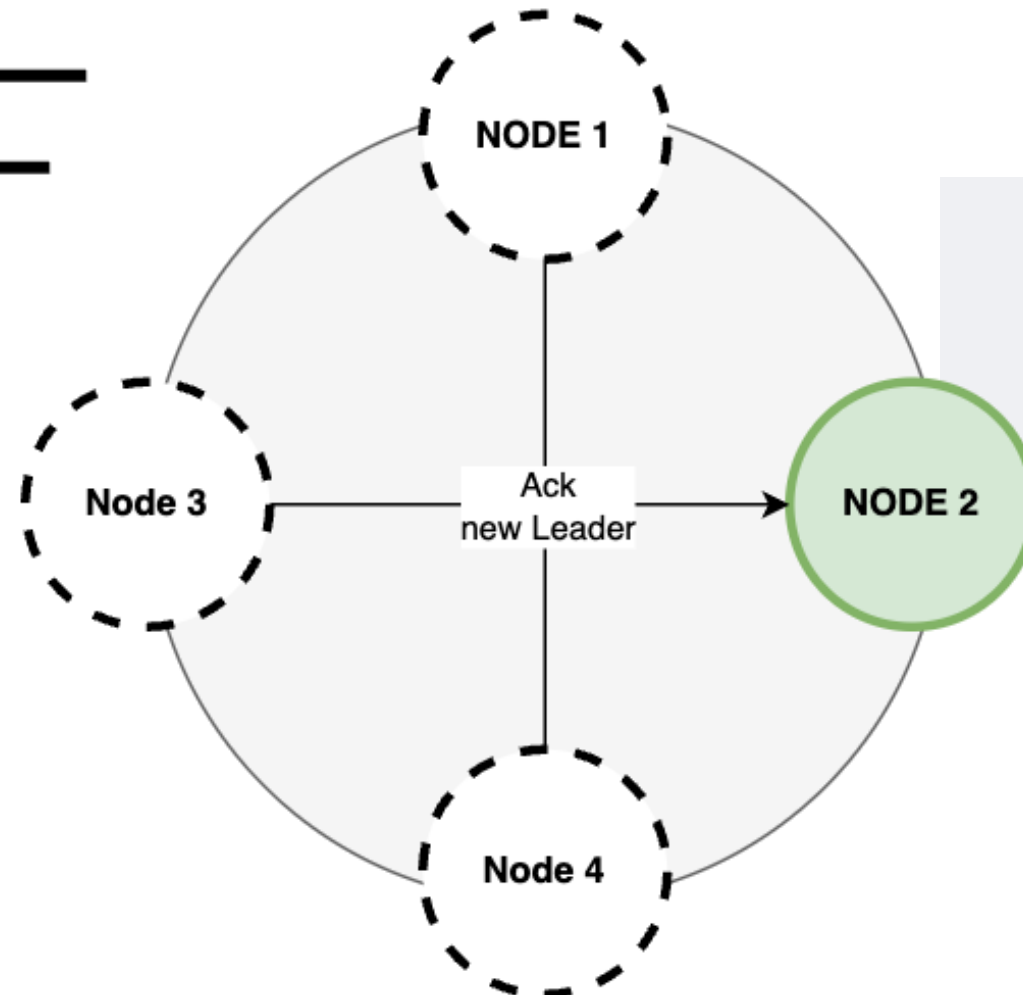
09 LEADER-ELECTION, WER IST DER BOSS? 🏰

Konsens-Algorithmus – Raft

Follower - - -

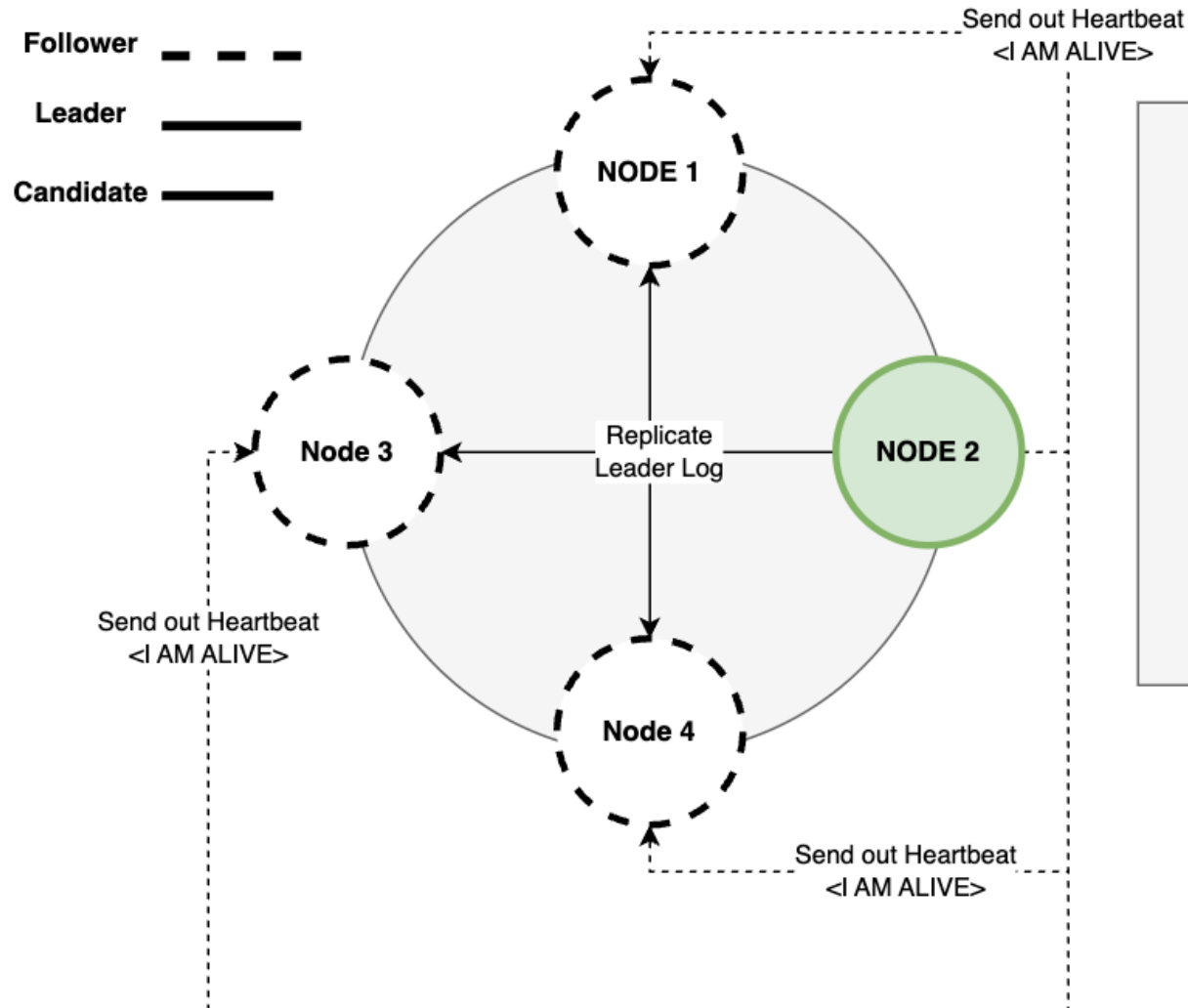
Leader ———

Candidate ———



09 LEADER-ELECTION, WER IST DER BOSS? 🏰

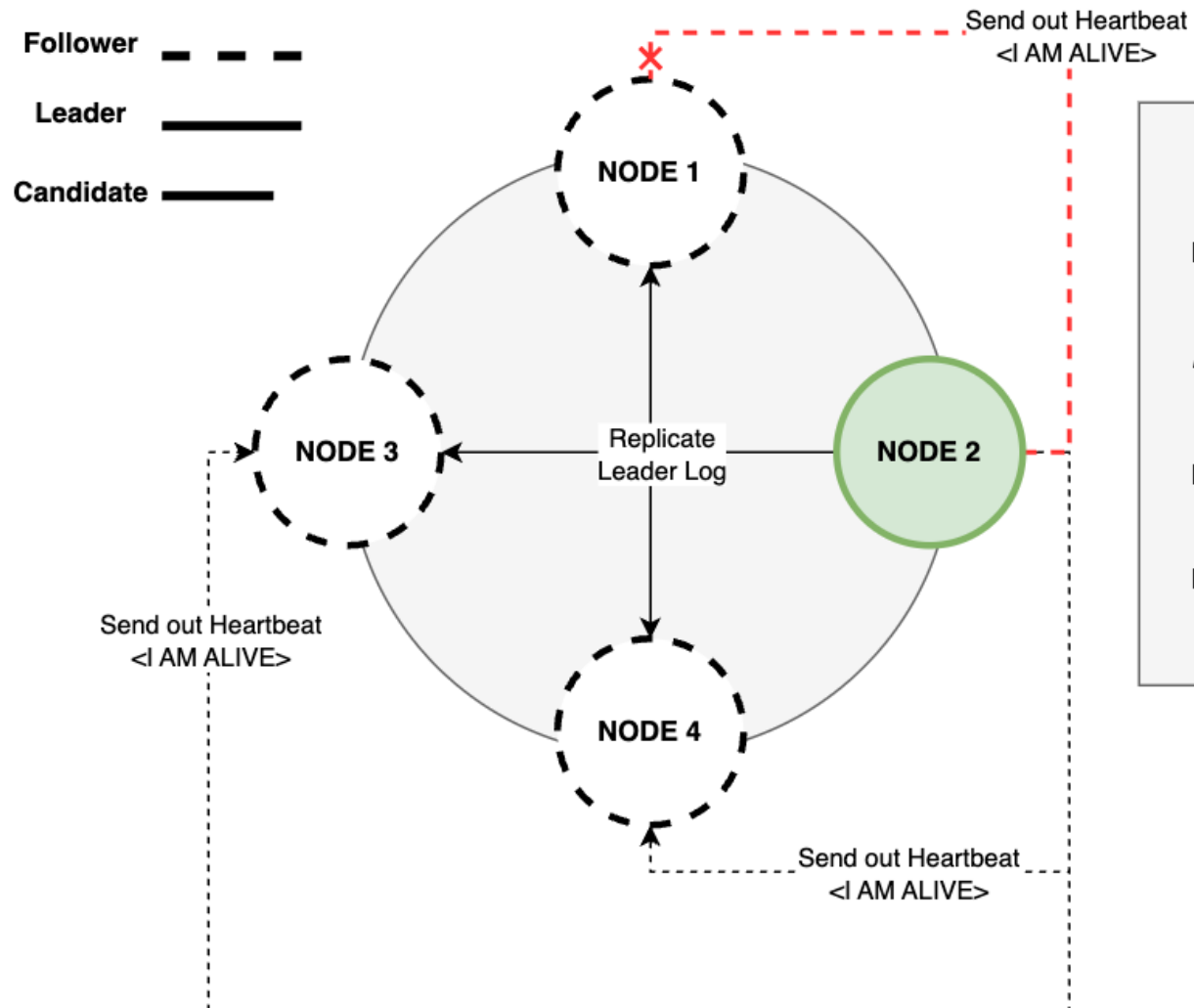
Konsens-Algorithmus – Raft



	1	2	3	4	...	10
N1	2	4	4			
N2	2	4	4			
N3	2	4	4			
N4	2	4	4			

09 LEADER-ELECTION, WER IST DER BOSS? 🏰

Konsens-Algorithmus – Raft

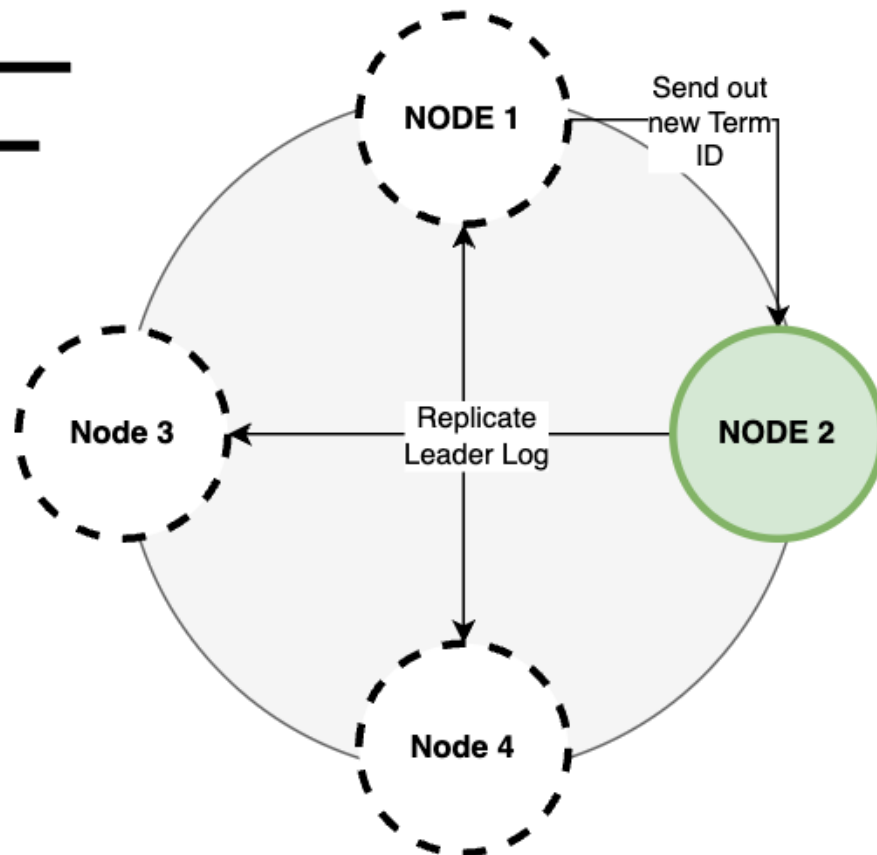


	1	2	3	4	...	10
N1	2	4	4			
N2	2	4	4			
N3	2	4	4			
N4	2	4	4			

09 LEADER-ELECTION, WER IST DER BOSS? 🏰

Konsens-Algorithmus – Raft

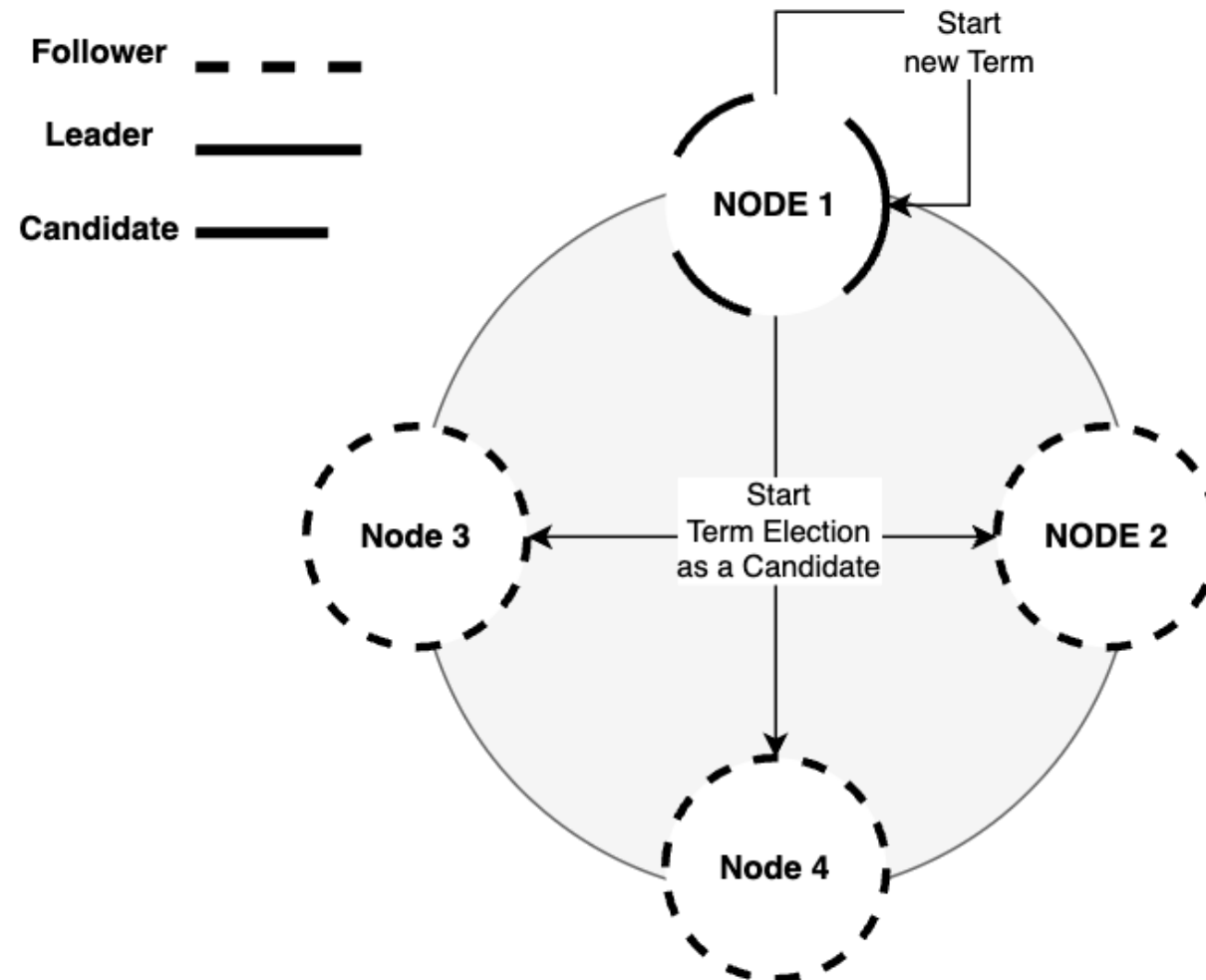
Follower - - -
Leader ———
Candidate ———



	1	2	3	4	...	10
N1	2	4	4	6		
N2	2	4	4			
N3	2	4	4			
N4	2	4	4			

09 LEADER-ELECTION, WER IST DER BOSS? 🏰

Konsens-Algorithmus – Raft



09 Leader-Election, wer ist der Boss? 🏰

***Für welchen
Algorithmus haben wir
uns entschieden? 🤔***

**und warum nicht für Raft?*

09 LEADER-ELECTION, WER IST DER BOSS? 🏰

Entscheidung

Es ist im Moment noch zu kompliziert ☐



09 Leader-Election, wer ist der Boss? 👑

Daher... Bully Election

DEMO TIME 

10

LEARNING LESSONS 🤔

10 LEARNING LESSONS 🤔

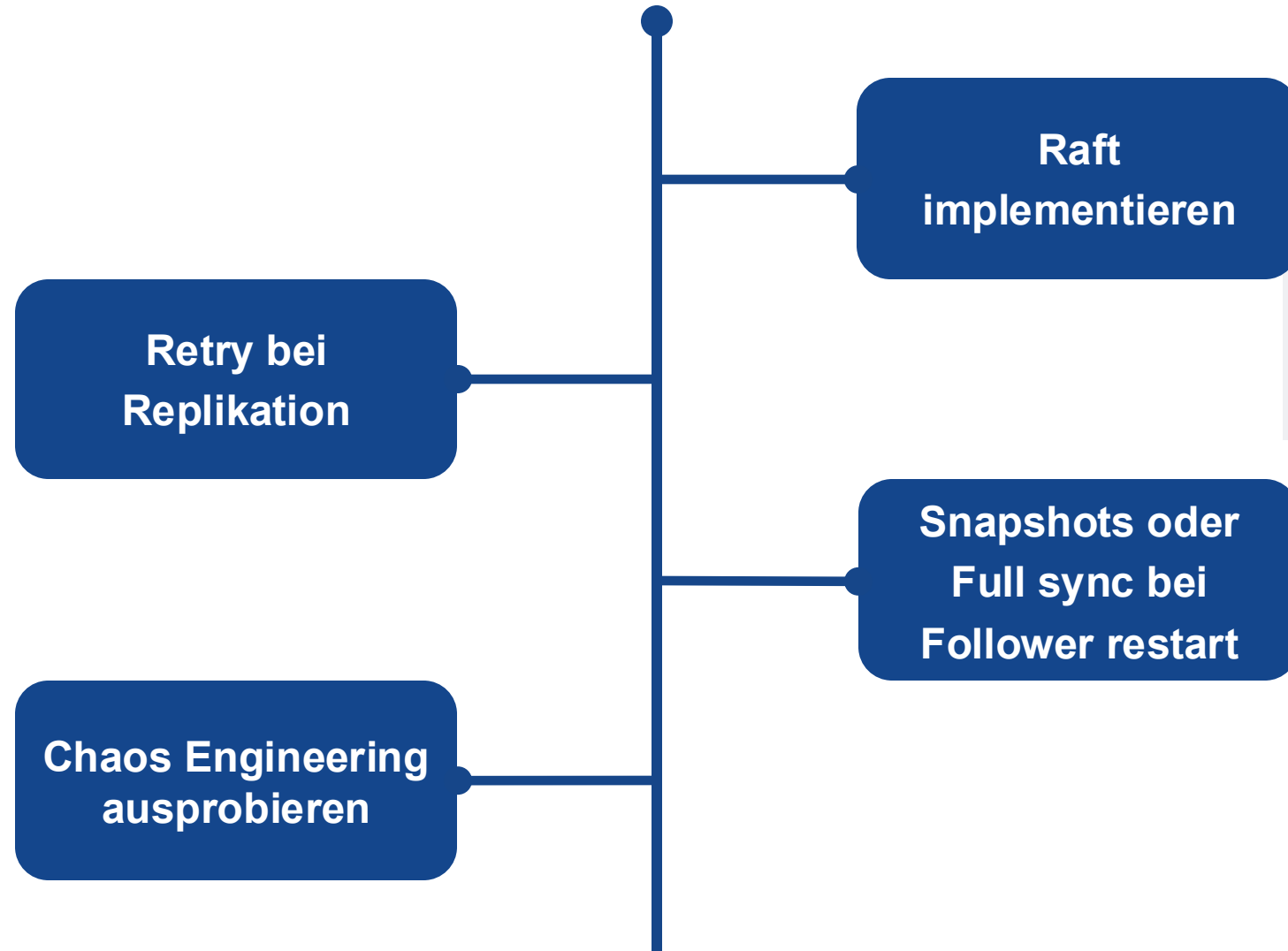
- **Netzwerkkommunikation ist sehr fehleranfällig**
/skill-issue
- **Leader-Election ist ein Rabbit-Hole an sich**
- **Einfache Systeme sind oft nicht einfach, sondern reduziert komplex**

11

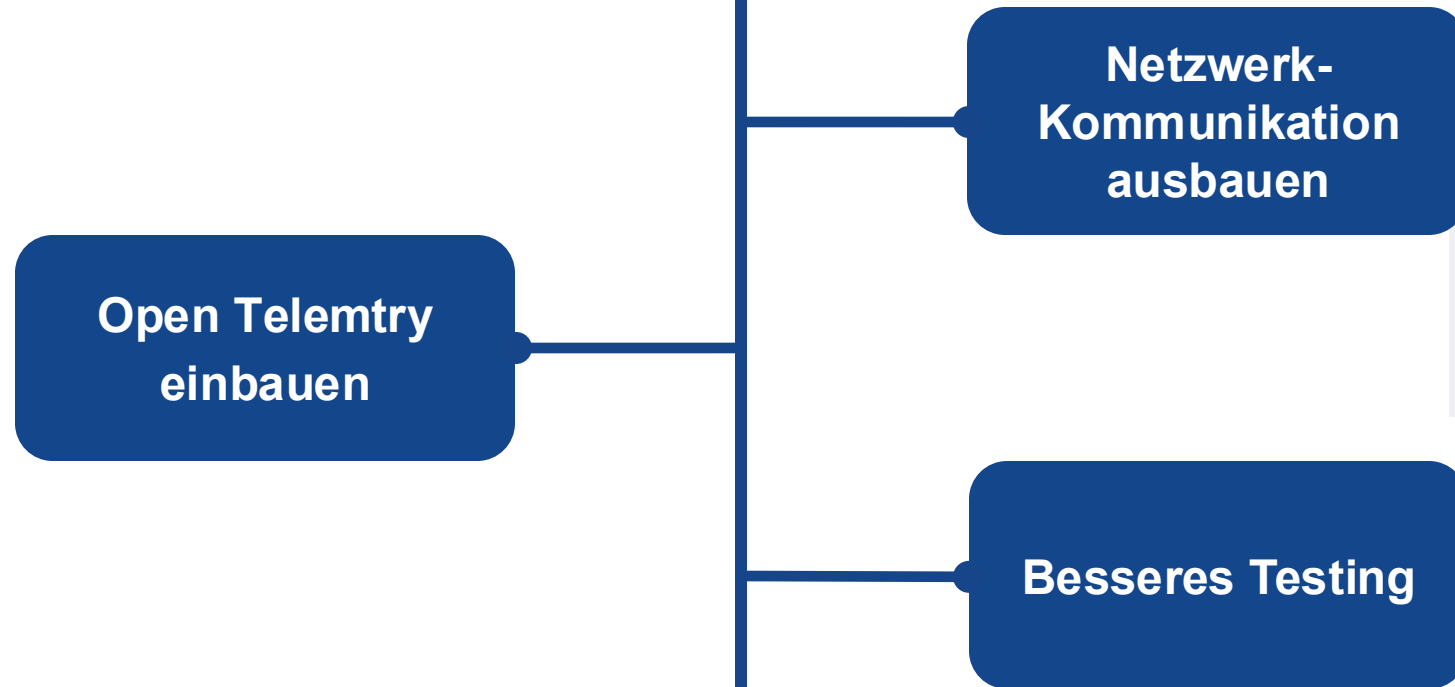
“ROADMAP”



11 “ROADMAP”



11 “ROADMAP”



Vielen Dank!

Feedback is appreciated!

<https://forms.office.com/e/hvKzZK5Mxm>

Feedback zur Präsentation - Kein
SQL, Kein ORM, Kein Schlaf – Ein
Key-Value-Store in C++

